

PHASE 3 — DOCKER & CONTAINERIZATION

DAY 10 — CONTAINERS FUNDAMENTALS

Topic: Containers Fundamentals

Tool: Docker

Objectives

- Understand Docker Images
- Understand Image Layers
- Understand Docker Registries
- Understand Containers vs Virtual Machines
- Understand Linux Namespaces
- Understand Linux cgroups

Hands-on Tasks

- Run containers
- Inspect containers
- Stop containers
- Remove containers
- Map ports
- Mount volumes
- Inspect image layers

1. Docker Images

A Docker image is a read-only template used to create containers.

An image contains the application, dependencies, libraries, and configuration required to run an application.

Key Points

- Images are used to create containers.
- Images are composed of multiple layers.

- Images can be stored in registries.
- One image can be used to create multiple containers.

Command Used

sudo docker pull nginx

sudo docker images

Screenshot

```
shayan@shayan-VirtualBox:~$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb6d261df4923378b419e0d24f0
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
shayan@shayan-VirtualBox:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	Info →	In Use
nginx:latest	b34848eff6db	241MB	66.2MB	U		

```
shayan@shayan-VirtualBox:~$
```

2. Image Layers

Docker images are built from multiple layers.

Each layer represents filesystem changes that are combined to form the final image.

Key Points

- Layers are stacked together.
- Layers can be reused.
- Docker can cache layers during image builds.
- Layer reuse can reduce storage and build time.

Command Used

sudo docker history nginx

```
shayan@shayan-VirtualBox:~$ sudo docker history nginx
```

IMAGE	COMMENT	CREATED	CREATED BY	SIZE
b34848eff6db		6 days ago	CMD ["nginx" "-g" "daemon off;"]	0B
<missing>	buildkit.dockerfile.v0	6 days ago	STOPSIGNAL SIGQUIT	0B
<missing>	buildkit.dockerfile.v0	6 days ago	EXPOSE map[80/tcp:{}]	0B
<missing>	buildkit.dockerfile.v0	6 days ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B
<missing>	buildkit.dockerfile.v0	6 days ago	COPY 30-tune-worker-processes.sh /docker-ent...	16.4 kB
<missing>	buildkit.dockerfile.v0	6 days ago	COPY 20-envsubst-on-templates.sh /docker-ent...	12.3 kB
<missing>	buildkit.dockerfile.v0	6 days ago	COPY 15-local-resolvers.envsh /docker-entryp...	12.3 kB
<missing>	buildkit.dockerfile.v0	6 days ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	12.3 kB

3. Docker Registries

A Docker registry is a service used to store and distribute Docker images.

The basic workflow is:

Docker Image



Push



Registry



Pull



Docker Host

A registry can be public or private.

Example

```
sudo docker pull nginx
```

The command downloads the Nginx image from a Docker registry.

4. Containers vs Virtual Machines

Containers and virtual machines both provide isolation, but they use different approaches.

Containers

Application



Container



Container Runtime



Host OS Kernel



Hardware

Containers share the host operating system kernel.

Virtual Machines

Application



Guest Operating System



Virtual Hardware



Hypervisor



Hardware

Virtual machines run a separate guest operating system.

Comparison

Feature	Containers	Virtual Machines
Kernel	Share host kernel	Separate guest OS
Resource usage	Generally lower	Generally higher
Startup	Usually faster	Usually slower
Size	Usually smaller	Usually larger
Isolation	Process/OS level	Hardware virtualization

Key Difference

Containers share the host kernel, while virtual machines run a separate guest operating system.

5. Linux Namespaces

Linux namespaces provide isolation between groups of processes.

Docker uses namespaces to isolate containers from each other and from the host.

Namespaces can isolate:

- Processes
- Networks
- Mount points
- Hostnames
- Users

Concept

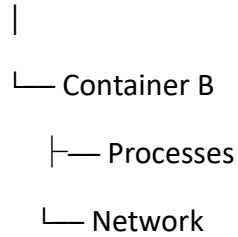
Host

|

| — Container A

| | — Processes

| | — Network



Key Point

Namespaces provide isolation.

6. Linux cgroups

cgroups, or control groups, are a Linux kernel feature used to control and monitor resource usage by groups of processes.

Docker uses cgroups for container resource management.

Resources include:

- CPU
- Memory
- Number of processes

Concept

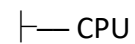
Container



cgroups



Resource Control



Key Point

cgroups provide resource control and accounting.

7. Hands-on Demonstration

7.1 Run a Container

Command

```
sudo docker run -d --name day10-web -p 8080:80 nginx
```

Verify

```
sudo docker ps
```

Result

The Nginx container runs in detached mode.

The host port 8080 is mapped to container port 80.

Screenshot



```
shayan@shayan-VirtualBox:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
	PORTS	NAMES		
55a632fb59a6	nginx	"/docker-entrypoint..."	About a minute ago	Up About a minute
	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	day10-web		

```
shayan@shayan-VirtualBox:~$
```

8. Test Port Mapping

Open the following address in a web browser:

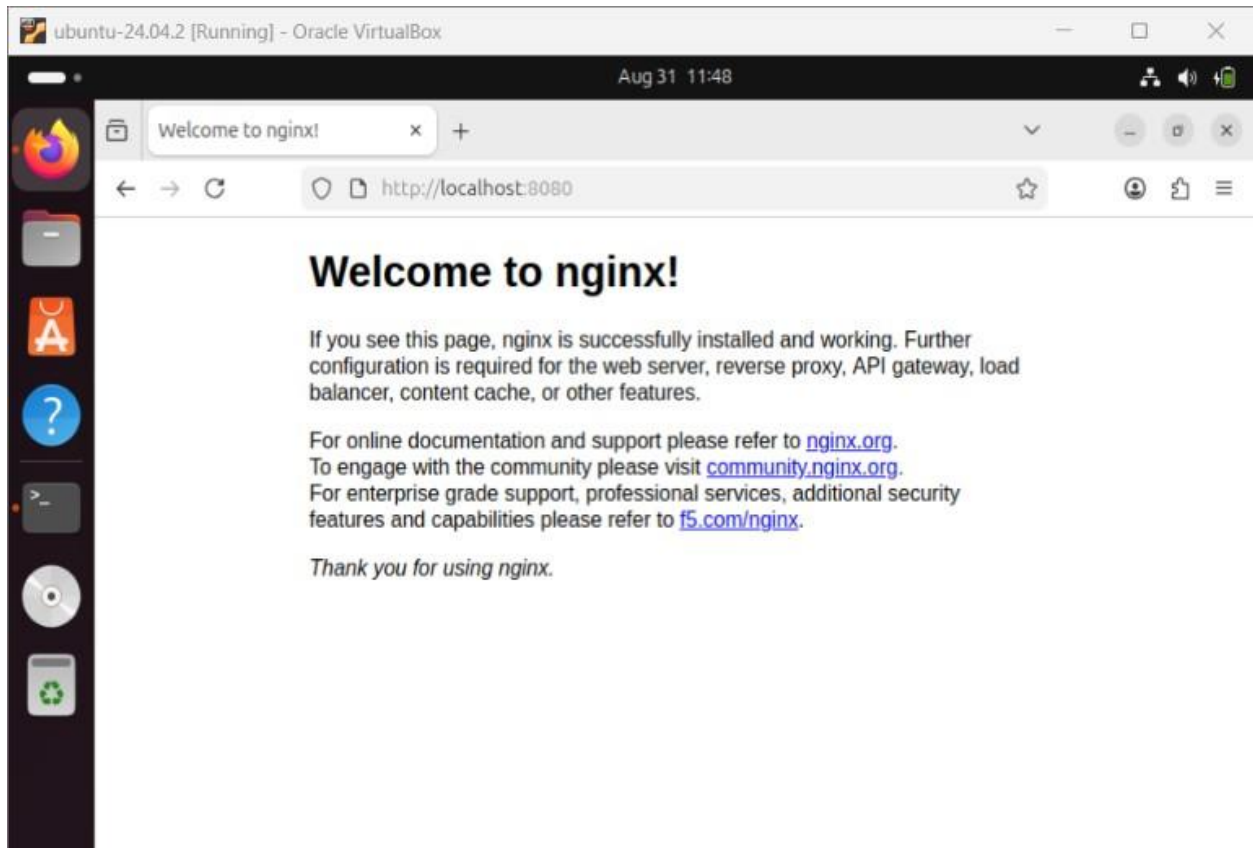
<http://localhost:8080>

Result

The Nginx welcome page is displayed.

This confirms that traffic from host port 8080 reaches port 80 inside the container.

Screenshot



9. Inspect the Container

Command

```
sudo docker inspect day10-web
```

A shorter version can be used to display important information:

```
sudo docker inspect --format '{{.Name}}'
```

```
Status: '{{.State.Status}}'
```

```
Image: '{{.Config.Image}}'
```

```
IP: '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' day10-web
```

Result

The command provides information about the container configuration, status, image, networking, and other settings.

Screenshot

```
shayan@shayan-VirtualBox:~$ sudo docker inspect --format 'Name: {{.Name}}
Status: {{.State.Status}}
Image: {{.Config.Image}}
IP: {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' day10-web
Name: /day10-web
Status: running
Image: nginx
IP: 172.17.0.2
shayan@shayan-VirtualBox:~$
```

10. Stop and Remove the Container

Command

```
sudo docker stop day10-web
```

```
sudo docker rm day10-web
```

Verify:

```
sudo docker ps -a
```

Result

The container changes from a running state to a stopped state.

Screenshot

```
shayan@shayan-VirtualBox:~$ sudo docker stop day10-web
shayan@shayan-VirtualBox:~$ sudo docker rm day10-web
shayan@shayan-VirtualBox:~$ sudo docker ps -a
day10-web
day10-web
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
shayan@shayan-VirtualBox:~$
```

11. Start the Container

Command

```
sudo docker start day10-web
```

Verify:

```
sudo docker ps
```

Result

The previously stopped container starts again.

This demonstrates the Docker container lifecycle.

Running



Stop



Stopped



Start



Running

12. Mount a Volume

Create a Docker volume:

```
sudo docker volume create day10-data
```

List volumes:

```
sudo docker volume ls
```

Run a container with the volume:

```
sudo docker run -d --name volume-test -v day10-data:/data nginx
```

Inspect the container:

```
sudo docker inspect volume-test
```

Result

The Docker volume day10-data is mounted at /data inside the container.

Screenshot

```
    "Mounts": [
      {
        "Type": "volume",
        "Name": "day10-data",
        "Source": "/var/snap/docker/common/var-lib-docker/volumes/day10-
data/_data",
        "Destination": "/data",
        "Driver": "local",
        "Mode": "z",
        "RW": true,
        "Propagation": ""
      }
    ],
    "Config": {
```

13. Inspect Image Layers

Command

sudo docker history nginx

Result

The command displays the history and layers associated with the Nginx image.

Screenshot

```
shayan@shayan-VirtualBox:~$ sudo docker history nginx
IMAGE          CREATED          CREATED BY          SIZE
COMMENT
b34848eff6db   6 days ago      CMD ["nginx" "-g" "daemon off;"] 0B
<missing>      6 days ago      STOPSIGNAL SIGQUIT              0B
<missing>      6 days ago      EXPOSE map[80/tcp:{}]           0B
<missing>      6 days ago      ENTRYPOINT ["/docker-entrypoint.sh"] 0B
<missing>      6 days ago      COPY 30-tune-worker-processes.sh /docker-ent... 16.4
kB    buildkit.dockerfile.v0
<missing>      6 days ago      COPY 20-envsubst-on-templates.sh /docker-ent... 12.3
kB    buildkit.dockerfile.v0
<missing>      6 days ago      COPY 15-local-resolvers.envsh /docker-entryp... 12.3
kB    buildkit.dockerfile.v0
<missing>      6 days ago      COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3
kB    buildkit.dockerfile.v0
```

14. Troubleshooting

Docker Permission Issue

During the setup, Docker initially returned:

permission denied while trying to connect to the Docker API

The Docker environment was investigated using:

```
docker --version
```

```
sudo systemctl status docker
```

The Docker installation was identified as a Snap installation.

Docker services were checked using:

```
sudo snap services docker
```

The Docker daemon was found to be active.

For the practical work, Docker commands were executed using:

```
sudo docker
```

Finding

The Docker CLI was installed and the Docker daemon was running, but the current user did not have direct permission to access the Docker socket.